Doxygen Assignment

1.0

Generated on Wed Nov 11 2026 22:29:34 for Doxygen Assignment by Doxygen 1.18.0

Wed Nov 11 2026 22:29:34

1 Doxygen Assignment 1
1.1 Introduction . . . 1
1.2 Features . . . 1
1.3 Usage . . . 1

2 Todo List 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 7
4.1 File List . . . 7

5 Class Documentation 9
5.1 CalculationEngine Class Reference . . . 9
5.1.1 Detailed Description . . . 9
5.1.2 Constructor & Destructor Documentation . . . 10
5.1.2.1 CalculationEngine() . . . 10
5.1.3 Member Function Documentation . . . 10
5.1.3.1 add() . . . 10
5.1.3.2 divide() . . . 10
5.1.3.3 multiply() . . . 11
5.1.3.4 power() . . . 11
5.1.3.5 subtract() . . . 12
5.1.4 Member Data Documentation . . . 12
5.1.4.1 logger . . . 12
5.2 Logger Class Reference . . . 13
5.2.1 Detailed Description . . . 13
5.2.2 Constructor & Destructor Documentation . . . 13
5.2.2.1 Logger() . . . 13
5.2.3 Member Function Documentation . . . 14
5.2.3.1 log() . . . 14
5.2.4 Member Data Documentation . . . 14
5.2.4.1 loggerName . . . 14

6 File Documentation 15
6.1 CalculationEngine.h File Reference . . . 15
6.1.1 Detailed Description . . . 15
6.2 CalculationEngine.h . . . 15
6.3 Logger.h File Reference . . . 16
6.3.1 Detailed Description . . . 17
6.3.2 Enumeration Type Documentation . . . 17
6.3.2.1 LogLevel . . . 17
6.4 Logger.h . . . 17

6.5 main.cpp File Reference 18
6.5.1 Detailed Description 18
6.5.2 Function Documentation 19
6.5.2.1 main() 19
6.6 main.cpp 19

Chapter 1

Doxygen Assignment

1.1 Introduction

This project demonstrates how Doxygen can be used to generate documentation for a multi-file C++ project.

1.2 Features

- Logging using different severity levels.
- Mathematical calculations.
- Documentation of classes and functions.

1.3 Usage

Compile and execute the program to see the mathematical operations and logging output.

Chapter 2

Todo List

Member CalculationEngine::power (double base, double exponent)

Add support for additional mathematical operations.

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CalculationEngine
: Performs mathematical calculations 9

Logger
: Provides logging functionality 13

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

CalculationEngine.h
: Contains the CalculationEngine class 15

Logger.h
: Contains the Logger class and logging levels 16

main.cpp
: Main application entry point 18

Chapter 5

Class Documentation

5.1 CalculationEngine Class Reference

Performs mathematical calculations.

`#include <CalculationEngine.h>`

Public Member Functions

- CalculationEngine ()
  Constructs a CalculationEngine object.
- double add (double a, double b)
  Adds two numbers.
- double subtract (double a, double b)
  Subtracts two numbers.
- double multiply (double a, double b)
  Multiplies two numbers.
- double divide (double a, double b)
  Divides two numbers.
- double power (double base, double exponent)
  Calculates the power of a number.

Private Attributes

- Logger logger

5.1.1 Detailed Description

Performs mathematical calculations.

This class provides basic mathematical operations such as addition, subtraction, multiplication, division and power calculation.

It also uses the Logger class to display information about calculation operations.

Definition at line 26 of file CalculationEngine.h.

5.1.2 Constructor & Destructor Documentation

5.1.2.1 CalculationEngine()

```
CalculationEngine::CalculationEngine ()  [inline]
```

Constructs a CalculationEngine object.

Definition at line 36 of file CalculationEngine.h.

```
00037         : logger("CalculationEngine")
00038     {
00039         logger.log(INFO, "Calculation Engine initialized.");
00040     }
```

5.1.3 Member Function Documentation

5.1.3.1 add()

```
double CalculationEngine::add (
            double a,
            double b)  [inline]
```

Adds two numbers.

Parameters

a	First number.
b	Second number.

Returns

Sum of the two numbers.

Definition at line 49 of file CalculationEngine.h.

```
00050     {
00051         logger.log(INFO, "Addition performed.");
00052         return a + b;
00053     }
```

5.1.3.2 divide()

```
double CalculationEngine::divide (
            double a,
            double b)  [inline]
```

Divides two numbers.

Parameters

a	Dividend.

b	Divisor.

Returns

Quotient of the division.

Definition at line 88 of file CalculationEngine.h.

```
00089     {
00090         if (b == 0)
00091         {
00092             logger.log(ERROR, "Division by zero attempted.");
00093             throw std::invalid_argument("Division by zero");
00094         }
00095 
00096         logger.log(INFO, "Division performed.");
00097         return a / b;
00098     }
```

5.1.3.3 multiply()

```
double CalculationEngine::multiply (
            double a,
            double b)  [inline]
```

Multiplies two numbers.

Parameters

a	First number.
b	Second number.

Returns

Product of the two numbers.

Definition at line 75 of file CalculationEngine.h.

```
00076     {
00077         logger.log(INFO, "Multiplication performed.");
00078         return a * b;
00079     }
```

5.1.3.4 power()

```
double CalculationEngine::power (
            double base,
            double exponent)  [inline]
```

Calculates the power of a number.

Parameters

base	The base number.

exponent	The exponent value.

Returns

Result of raising base to exponent.

Todo Add support for additional mathematical operations.

Definition at line 109 of file CalculationEngine.h.

```
00110     {
00111         logger.log(INFO, "Power calculation performed.");
00112         return std::pow(base, exponent);
00113     }
```

5.1.3.5 subtract()

```
double CalculationEngine::subtract (
            double a,
            double b)  [inline]
```

Subtracts two numbers.

Parameters

a	First number.
b	Second number.

Returns

Difference between the two numbers.

Definition at line 62 of file CalculationEngine.h.

```
00063     {
00064         logger.log(INFO, "Subtraction performed.");
00065         return a - b;
00066     }
```

5.1.4 Member Data Documentation

5.1.4.1 logger

```
Logger CalculationEngine::logger  [private]
```

Definition at line 29 of file CalculationEngine.h.

The documentation for this class was generated from the following file:

- CalculationEngine.h

5.2 Logger Class Reference

Provides logging functionality.

`#include <Logger.h>`

Public Member Functions

- Logger (const string &name)
  Constructs a Logger object.
- void log (LogLevel level, const string &message)
  Displays a log message.

Private Attributes

- string loggerName

5.2.1 Detailed Description

Provides logging functionality.

The Logger class displays messages with their corresponding logging levels.

Definition at line 38 of file Logger.h.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 Logger()

```
Logger::Logger (
            const string & name)  [inline]
```

Constructs a Logger object.

Parameters

name	Name assigned to the logger.

Definition at line 50 of file Logger.h.

```
00051     {
00052         loggerName = name;
00053     }
```

5.2.3 Member Function Documentation

5.2.3.1 log()

```
void Logger::log (
            LogLevel level,
            const string & message)  [inline]
```

Displays a log message.

Parameters

level	Severity level of the message.
message	Message to be displayed.

Definition at line 61 of file Logger.h.

```
00062     {
00063         string levelName;
00064 
00065         switch (level)
00066         {
00067             case DEBUG:
00068                 levelName = "DEBUG";
00069                 break;
00070 
00071             case INFO:
00072                 levelName = "INFO";
00073                 break;
00074 
00075             case WARNING:
00076                 levelName = "WARNING";
00077                 break;
00078 
00079             case ERROR:
00080                 levelName = "ERROR";
00081                 break;
00082 
00083             default:
00084                 levelName = "UNKNOWN";
00085         }
00086 
00087         cout « "[" « levelName « "] "
00088                   « loggerName « ": "
00089                   « message « endl;
00090     }
```

5.2.4 Member Data Documentation

5.2.4.1 loggerName

```
string Logger::loggerName  [private]
```

Definition at line 41 of file Logger.h.

The documentation for this class was generated from the following file:

- Logger.h

Chapter 6

File Documentation

6.1 CalculationEngine.h File Reference

Contains the CalculationEngine class.

```
#include "Logger.h"
#include <cmath>
#include <stdexcept>
```

Classes

- class CalculationEngine

  Performs mathematical calculations.

6.1.1 Detailed Description

Contains the CalculationEngine class.

Definition in file CalculationEngine.h.

6.2 CalculationEngine.h

Go to the documentation of this file.

```
00001
00006
00007 #ifndef CALCULATION_ENGINE_H
00008 #define CALCULATION_ENGINE_H
00009
00010 #include "Logger.h"
00011 #include <cmath>
00012 #include <stdexcept>
00013
00026 class CalculationEngine
00027 {
00028 private:
00029     Logger logger;
00030
00031 public:
```

```
00032
00036     CalculationEngine()
00037         : logger("CalculationEngine")
00038     {
00039         logger.log(INFO, "Calculation Engine initialized.");
00040     }
00041
00049     double add(double a, double b)
00050     {
00051         logger.log(INFO, "Addition performed.");
00052         return a + b;
00053     }
00054
00062     double subtract(double a, double b)
00063     {
00064         logger.log(INFO, "Subtraction performed.");
00065         return a - b;
00066     }
00067
00075     double multiply(double a, double b)
00076     {
00077         logger.log(INFO, "Multiplication performed.");
00078         return a * b;
00079     }
00080
00088     double divide(double a, double b)
00089     {
00090         if (b == 0)
00091         {
00092             logger.log(ERROR, "Division by zero attempted.");
00093             throw std::invalid_argument("Division by zero");
00094         }
00095
00096         logger.log(INFO, "Division performed.");
00097         return a / b;
00098     }
00099
00109     double power(double base, double exponent)
00110     {
00111         logger.log(INFO, "Power calculation performed.");
00112         return std::pow(base, exponent);
00113     }
00114 };
00115
00116 #endif
```

6.3 Logger.h File Reference

Contains the Logger class and logging levels.

```
#include <iostream>
#include <string>
```

Classes

- class Logger

  Provides logging functionality.

Enumerations

- enum LogLevel { DEBUG , INFO , WARNING , ERROR }

  Defines the available logging levels.

6.3.1 Detailed Description

Contains the Logger class and logging levels.

This file defines a simple logging system that can display messages with different severity levels.

Definition in file Logger.h.

6.3.2 Enumeration Type Documentation

6.3.2.1 LogLevel

```
enum LogLevel
```

Defines the available logging levels.

The logging level determines the importance and severity of a log message.

Enumerator

DEBUG	Detailed debugging information.
INFO	General information messages.
WARNING	Messages indicating potential problems.
ERROR	Messages indicating errors.

Definition at line 24 of file Logger.h.

```
00025 {
00026     DEBUG,
00027     INFO,
00028     WARNING,
00029     ERROR
00030 };
```

6.4 Logger.h

Go to the documentation of this file.

```
00001
00009
00010 #ifndef LOGGER_H
00011 #define LOGGER_H
00012
00013 #include <iostream>
00014 #include <string>
00015 using namespace std ;
00016
00024 enum LogLevel
00025 {
00026     DEBUG,
00027     INFO,
00028     WARNING,
00029     ERROR
00030 };
00031
00038 class Logger
00039 {
00040 private:
00041     string loggerName;
00042
00043 public:
00044
```

```
00050     Logger(const string& name)
00051     {
00052         loggerName = name;
00053     }
00054
00061     void log(LogLevel level, const string& message)
00062     {
00063         string levelName;
00064
00065         switch (level)
00066         {
00067             case DEBUG:
00068                 levelName = "DEBUG";
00069                 break;
00070
00071             case INFO:
00072                 levelName = "INFO";
00073                 break;
00074
00075             case WARNING:
00076                 levelName = "WARNING";
00077                 break;
00078
00079             case ERROR:
00080                 levelName = "ERROR";
00081                 break;
00082
00083             default:
00084                 levelName = "UNKNOWN";
00085         }
00086
00087         cout « "[" « levelName « "] "
00088                   « loggerName « ": "
00089                   « message « endl;
00090     }
00091 };
00092
00093 #endif
```

6.5 main.cpp File Reference

Main application entry point.

```
#include <iostream>
#include "Logger.h"
#include "CalculationEngine.h"
```

Functions

- int main ()

  Main function of the application.

6.5.1 Detailed Description

Main application entry point.

Definition in file main.cpp.

6.5.2 Function Documentation

6.5.2.1 main()

```
int main ()
```

Main function of the application.

Creates a CalculationEngine object and performs different mathematical operations.

Returns

0 if the program executes successfully.

Definition at line 34 of file main.cpp.

```
00035 {
00036     Logger logger("Main");
00037     CalculationEngine calculator;
00038
00039     logger.log(INFO, "Starting application.");
00040
00041     double a = 10;
00042     double b = 5;
00043
00044     std::cout « "\nAddition: "
00045               « calculator.add(a, b) « std::endl;
00046
00047     std::cout « "Subtraction: "
00048               « calculator.subtract(a, b) « std::endl;
00049
00050     std::cout « "Multiplication: "
00051               « calculator.multiply(a, b) « std::endl;
00052
00053     std::cout « "Division: "
00054               « calculator.divide(a, b) « std::endl;
00055
00056     std::cout « "Power: "
00057               « calculator.power(a, b) « std::endl;
00058
00059     logger.log(INFO, "Application completed.");
00060
00061     return 0;
00062 }
```

6.6 main.cpp

Go to the documentation of this file.

```
00001
00021
00022 #include <iostream>
00023 #include "Logger.h"
00024 #include "CalculationEngine.h"
00025
00034 int main()
00035 {
00036     Logger logger("Main");
00037     CalculationEngine calculator;
00038
00039     logger.log(INFO, "Starting application.");
00040
00041     double a = 10;
00042     double b = 5;
00043
00044     std::cout « "\nAddition: "
00045               « calculator.add(a, b) « std::endl;
00046
00047     std::cout « "Subtraction: "
00048               « calculator.subtract(a, b) « std::endl;
00049
00050     std::cout « "Multiplication: "
00051               « calculator.multiply(a, b) « std::endl;
```

```
00052
00053     std::cout « "Division: "
00054               « calculator.divide(a, b) « std::endl;
00055
00056     std::cout « "Power: "
00057               « calculator.power(a, b) « std::endl;
00058
00059     logger.log(INFO, "Application completed.");
00060
00061     return 0;
00062 }
```